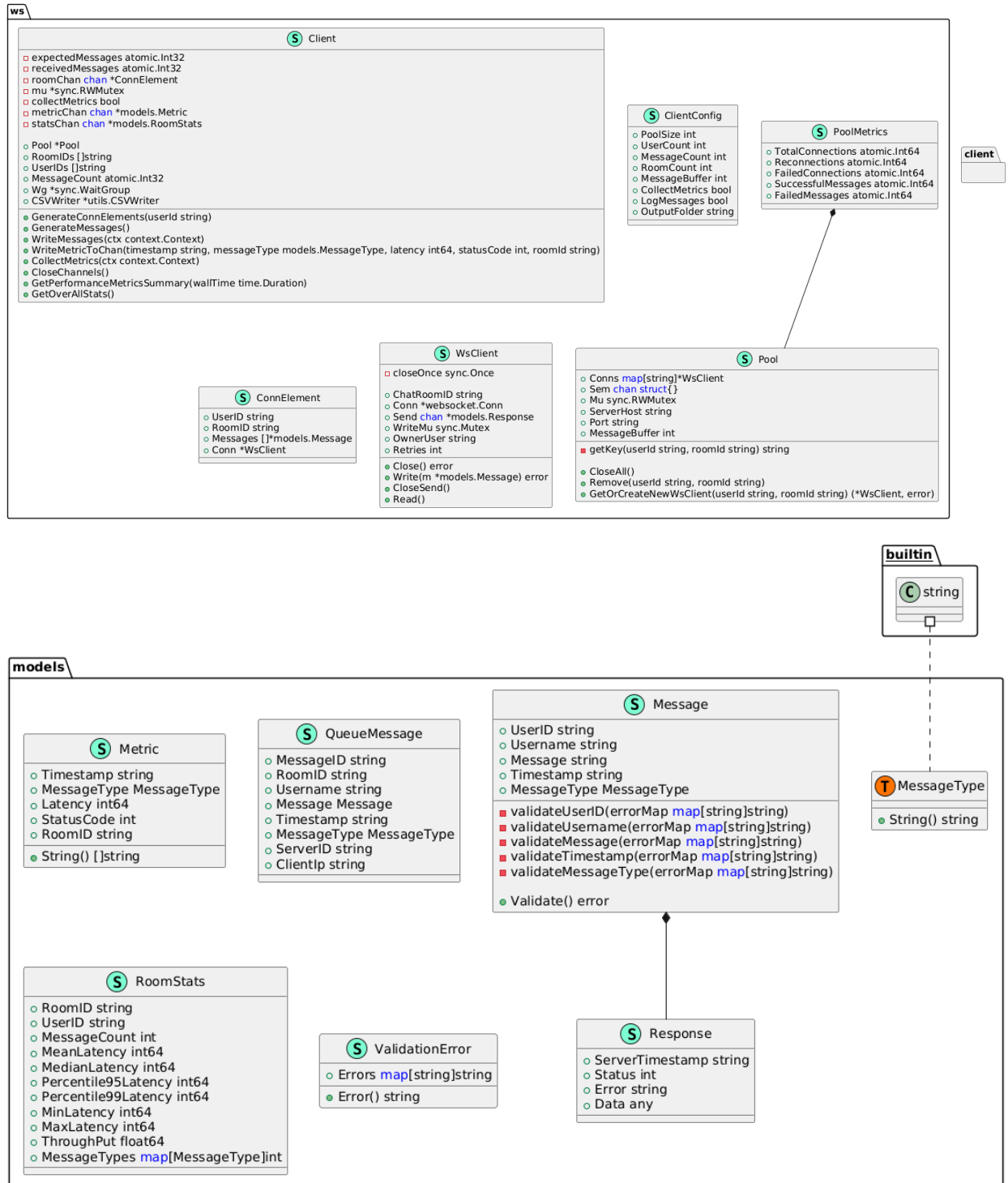
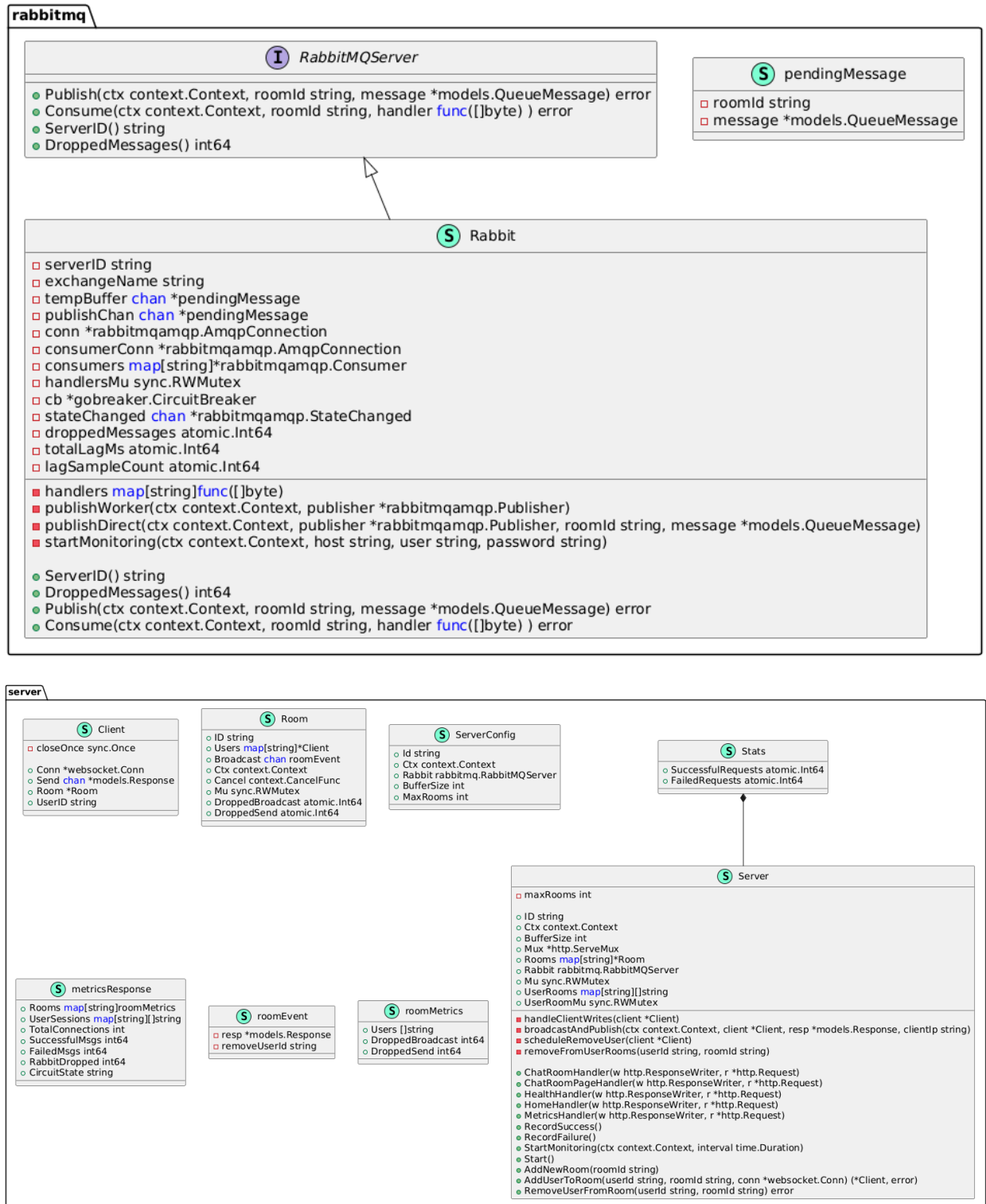


1. **Github link:** <https://github.khoury.northeastern.edu/supriyakotturu/chat-flow>
2. **ClassDiagrams:**



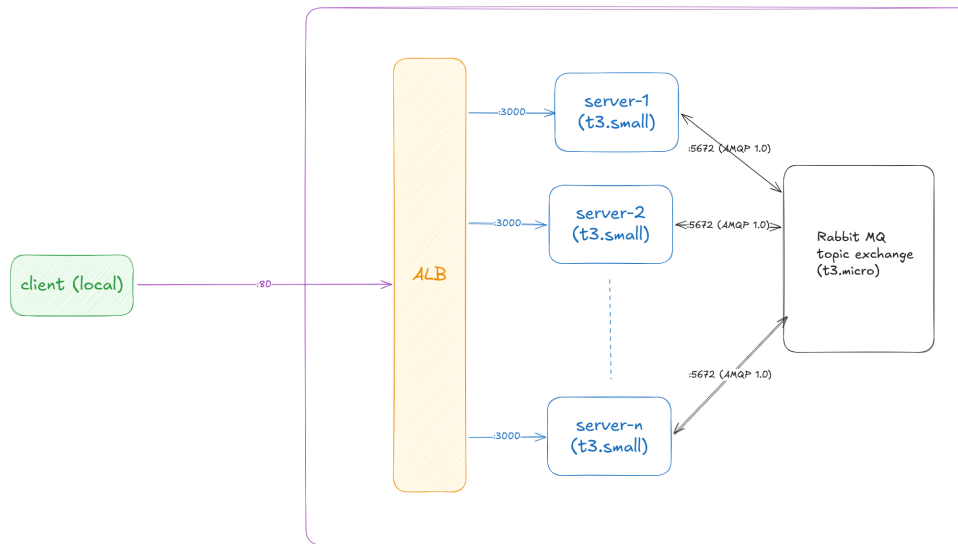


3. Server Architecture Diagram:

Each chat server maintains:

- WebSocket connections to all clients assigned to it by the ALB
- One AMQP publisher connection (20 publish workers)

c. One AMQP consumer connection (N consumers, one per room)



Why I chose this architecture:

Each chat server acts as both a RabbitMQ producer and consumer. This is not purely a design choice - it is structurally required by the architecture. WebSocket connections are stateful and bound to a specific server, so each server must consume from the fan-out exchange to deliver messages to its locally connected clients. There is no mechanism to forward a WebSocket write across servers without a separate routing layer.

Co-location also eliminates the need for an external state to track which clients are on which server. With fan-out semantics, every server receives every message and delivers it to whoever is locally connected - no stale routing entries when clients reconnect, no additional failure domains, no added infrastructure cost.

The failure model is equally simple. If a server dies, its clients disconnect and reconnect through the load balancer to a healthy node. There is no orphaned consumer state to recover.

The honest tradeoff is that each server consumes messages for rooms it may have no local clients in. At the current scale - 2 servers, 20 rooms - this overhead is negligible. Testing confirmed this: running the same workload across 2 servers instead of 1 showed no meaningful throughput gain (-1.8% at Pool=1000, +5.6% at Pool=64). If the servers were the bottleneck, doubling them would produce roughly 2x throughput. The flat results indicate the bottleneck is RabbitMQ fan-out throughput, not per-server compute.

Fan-out also means horizontal scaling works against itself at the broker level. Each additional server becomes another publisher pushing messages and another consumer pulling all of them, so RabbitMQ's load grows linearly with the number of servers while the throughput gain per server shrinks. Meaningful horizontal scaling would require either a RabbitMQ

cluster or switching from fan-out to topic/direct routing so each server only processes messages relevant to its own clients — a significantly more complex architecture that is not warranted at this scale.

The current design meets the assignment's requirements and leaves that evolution as a natural next step if load requirements change.

--

Each chat server acts as both a RabbitMQ producer and consumer. This is not purely a design choice - it is structurally required by the architecture. WebSocket connections are stateful and bound to a specific server, so each server must consume from the fan-out exchange to deliver messages to its locally connected clients. There is no mechanism to forward a WebSocket write across servers without introducing a separate routing layer.

Co-location (having producer and consumer in the same node) also eliminates the need for external state to track which clients are on which server. With fan-out semantics, every server receives every message and delivers it to whoever is locally connected. This removes a class of consistency problems: no stale routing entries when clients reconnect, no additional failure domains, and no added infrastructure cost.

The failure model is also simpler. If a server dies, its clients disconnect and reconnect through the load balancer to a healthy node. There is no orphaned consumer state to recover.

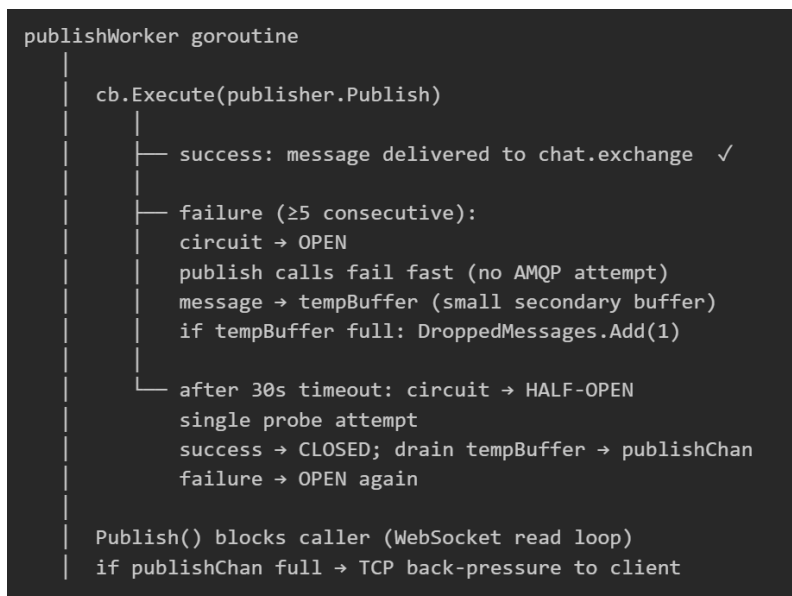
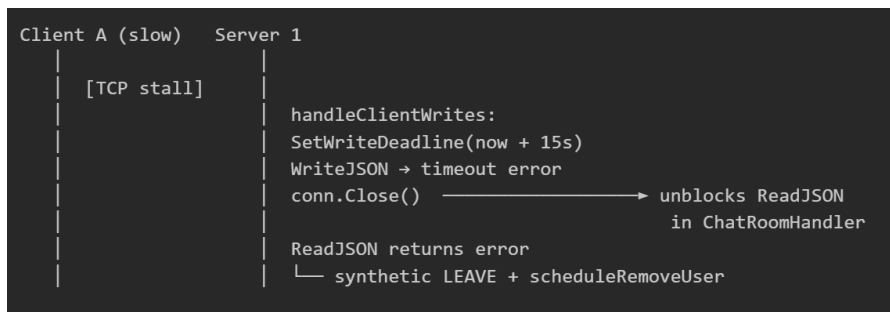
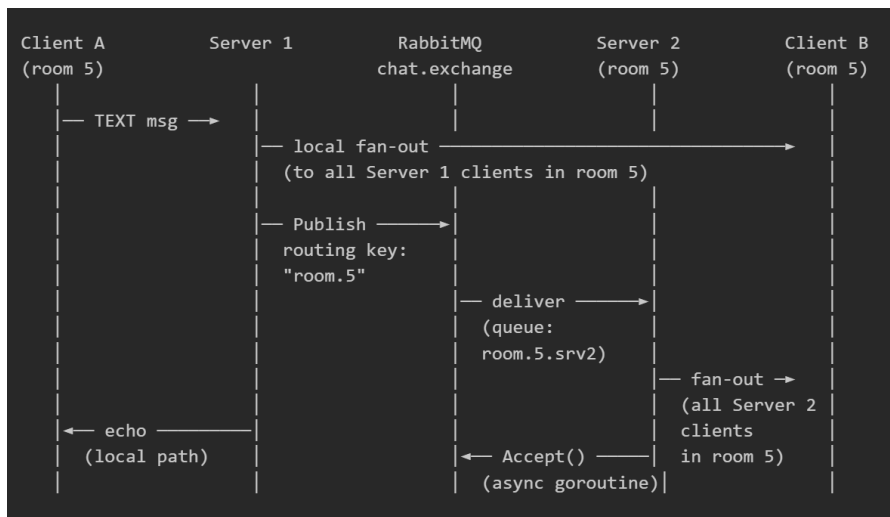
The honest **tradeoff** is that each server consumes messages for rooms it may have no clients in. At the current scale - 2 servers, 20 rooms - this overhead is negligible. **When I doubled the servers (1 -> 2), throughput barely changed.** If the server CPU/memory were the bottleneck, you'd expect ~2x throughput from 2 servers. That means the servers were never the bottleneck to begin with — RabbitMQ was. So making the servers more sophisticated (adding routing layers, separate services) wouldn't have helped anyway.

Running the same workload across 2 servers instead of 1 showed no meaningful throughput gain (-1.8% at Pool=1000, +5.6% at Pool=64), confirming that the bottleneck is RabbitMQ fan-out throughput, not per-server compute. Adding architectural complexity to address a bottleneck that does not exist at this scale would be premature.

Because of fan-out, adding more servers actually makes the RabbitMQ bottleneck worse, not better. Each new server added becomes another consumer pulling all messages AND another publisher pushing messages. So RabbitMQ's load grows linearly with the number of servers, while the throughput gain per server shrinks.

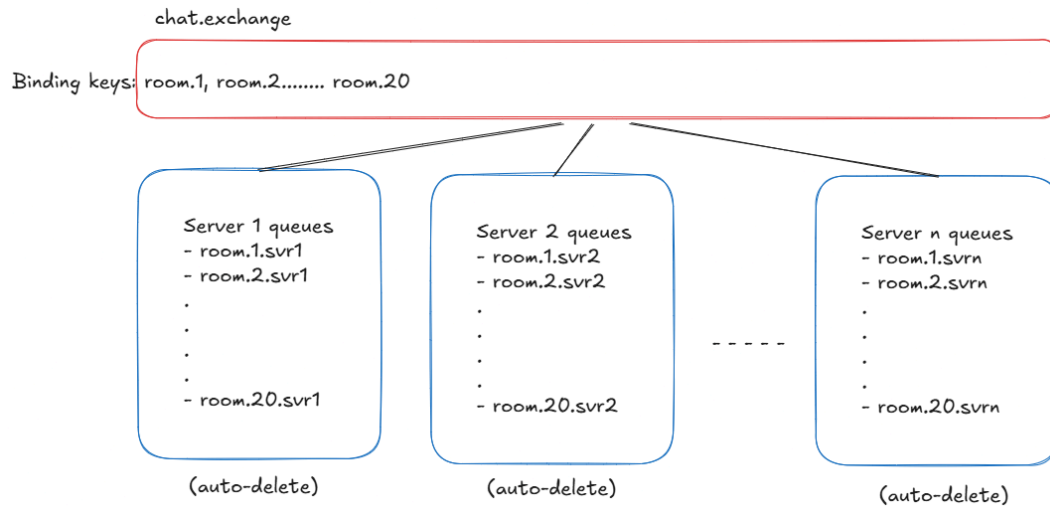
The only way horizontal scaling would genuinely help is if we also scaled RabbitMQ - e.g., a RabbitMQ cluster, or switching from fan-out to topic/direct routing so each server only handles messages for its own clients. But that's a much more complex architecture.

4. Message flow sequence diagram:



5. Queue Topology design:

Queue naming: room.<roomId>.<serverId>



Queues are declared with `IsAutoDelete: true`. When the server disconnects (and its AMQP consumer connection closes), all its queues are deleted automatically. This prevents stale queue accumulation across restarts.

Fan-out: A single message published to `chat.exchange` with routing key `room.5` is delivered to `N` queues (one per server). With 4 servers and 20 rooms, there are 80 queues total.

6. Consumer threading model:

```
NewRabbitMQ() (called at server startup)
├── consumerConn (AMQP connection - consumers only)
├── for each room 1..N:
│   └── go consumerLoop(consumer, roomId)      ← N goroutines, run for server lifetime
│       ┌── loop:
│       │   └── delivery, err = consumer.Receive(ctx) ← blocks until message arrives
│       │       └── handlersMu.RLock()
│       │           h = handlers[roomId]          ← handler registered later by Consume()
│       │           handlersMu.RUnlock()
│       │       └── measure consumer lag          ← always runs (even if h == nil)
│       │           parse Timestamp from raw JSON
│       │           totalLagMs.Add(time.Since(t).Ms())
│       │           lagSampleCount.Add(1)
│       │       └── if h != nil: h(data)          ← calls consumeMessages() in server.go
│       │           └── unmarshal QueueMessage
│       │               if ServerID == myID: skip ← self-filter
│       │               else: room.Broadcast <- event ← enqueue for local fan-out
│       │       └── go d.Accept(ctx)              ← async: return credit to broker immediately
│       │                                           (does NOT block the receive loop)
├── broadcast goroutine (one per room, in server.go)
│   └── loop:
│       └── event = <-room.Broadcast
│           └── if event.resp != nil:
│               └── fan out to all room.Users[*].Send channels
│               └── if event.removeUserId != "":
│                   └── close Send channel + delete from Users map (in-order removal)
│           └── <-room.Ctx.Done():
│               └── user.Conn.Close() ← unblocks ReadJSON in handler goroutines
│               └── close(user.Send) ← signals writer goroutine to exit
└── return
```

7. Load balancing configurations:

ALB listens on port 80 (standard HTTP, no root privileges required on the client side). Chat servers listen on port 3000 .

ALB port: 80 (HTTP)

Target group: chatflow-servers (port 3000, HTTP)

Healthcheck: GET /health

Stickiness: enabled (1 day, LB cookies). The AWSALB cookie pins each client to one EC2 server for 1 day.

8. Failure Handling strategies:

- a. *Backpressure*: If the 30 publish workers cannot drain ``publishChan`` fast enough, ``Publish()`` blocks the calling WebSocket read goroutine. This propagates back-pressure through the TCP stack to the client, slowing its send rate naturally. The alternative (dropping messages when the channel is full) was dropped because this allows the message to get broadcasted to the users connected in the server, but the users in other servers don't receive the message
- b. *Exponential back-off*: The client retries exponentially on connection errors and write errors to the chat server.
- c. *In-order Leave problem*: When a user leaves, we need to stop sending them messages. The naive approach is to close their Send channel immediately when the LEAVE arrives. But the broadcast goroutine is concurrently iterating room.Users and writing to Send channels. Closing Send mid-iteration causes either missed messages or a panic (send on closed channel). Instead of closing Send directly, the LEAVE handler enqueues a special removal event into the room's Broadcast channel using ``roomEvent`` struct.
 - i. RoomEvent populates ``removeUserId`` when there's a LEAVE message or connection errors. Else, it populates ``resp`` with the server response.
 - ii. Having this synthetic event helped to schedule removing the user, only after messages are broadcasted.
- d. *Circuit breaker*: The ``Rabbit`` struct uses the ``gobreaker`` library to wrap AMQP publish calls. ``gobreaker`` provides a standard three-state circuit breaker with configurable failure thresholds and automatic half-open probing.

9. Test Results:

Detailed test results can be found in [chatflow-assignment-2-test-results.pdf](#):

EC2 direct | 1000 pool size | 20 consumers | 500K messages | Async | initial credit 1000



```
=== Performance Metrics ===
Total runtime (wall time): 241.6s
Successful messages sent: 502000
Failed messages: 0
Overall throughput: 2077.6 msg/s
Total connections: 1000
Reconnections: 0
Failed connections: 0
Mean latency across all rooms: 116142ms
Median latency across all rooms: 118162ms
95th Percentile latency across all rooms: 120499ms
99th Percentile latency across all rooms: 133478ms
Min latency across all rooms: 1387ms
Max latency across all rooms: 143831ms
Median Throughput across all rooms: 107.01 msg/s
Throughput across rooms:

Room 19 | users: 50 | throughput: 5368.5 msg/s | mean latency: 115415ms | median latency: 117012ms
Message Types: JOIN: 1275 TEXT: 1221537 : 49 LEAVE: 1225

Room 12 | users: 50 | throughput: 5391.7 msg/s | mean latency: 120895ms | median latency: 121465ms
Message Types: TEXT: 1231185 : 49 LEAVE: 1225 JOIN: 1276

Room 13 | users: 50 | throughput: 5389.2 msg/s | mean latency: 118664ms | median latency: 118054ms
Message Types: TEXT: 1242224 LEAVE: 1275 JOIN: 1276

Room 11 | users: 50 | throughput: 5378.0 msg/s | mean latency: 121772ms | median latency: 121847ms
Message Types: JOIN: 1276 TEXT: 1242204 LEAVE: 1275

Room 3 | users: 50 | throughput: 5361.1 msg/s | mean latency: 121854ms | median latency: 121761ms
Message Types: JOIN: 1275 TEXT: 1244159 LEAVE: 1275

Room 4 | users: 50 | throughput: 5346.3 msg/s | mean latency: 114134ms | median latency: 116403ms
Message Types: : 96 LEAVE: 1176 TEXT: 1192627 JOIN: 1275

Room 17 | users: 50 | throughput: 5328.7 msg/s | mean latency: 121804ms | median latency: 122798ms
Message Types: JOIN: 1275 TEXT: 1219994 : 49 LEAVE: 1225

Room 1 | users: 50 | throughput: 5387.1 msg/s | mean latency: 117380ms | median latency: 118275ms
Message Types: LEAVE: 1176 JOIN: 1275 TEXT: 1194807 : 97

Room 18 | users: 50 | throughput: 5274.5 msg/s | mean latency: 116386ms | median latency: 118859ms
Message Types: JOIN: 1275 TEXT: 1186264 : 49 LEAVE: 1176

Room 20 | users: 50 | throughput: 5354.9 msg/s | mean latency: 119995ms | median latency: 121056ms
Message Types: : 49 JOIN: 1275 TEXT: 1224585 LEAVE: 1225

Room 7 | users: 50 | throughput: 5398.4 msg/s | mean latency: 116105ms | median latency: 116081ms
Message Types: JOIN: 1276 TEXT: 1241756 LEAVE: 1275

Room 5 | users: 50 | throughput: 5213.5 msg/s | mean latency: 112497ms | median latency: 116639ms
Message Types: : 143 LEAVE: 1128 JOIN: 1269 TEXT: 1162254

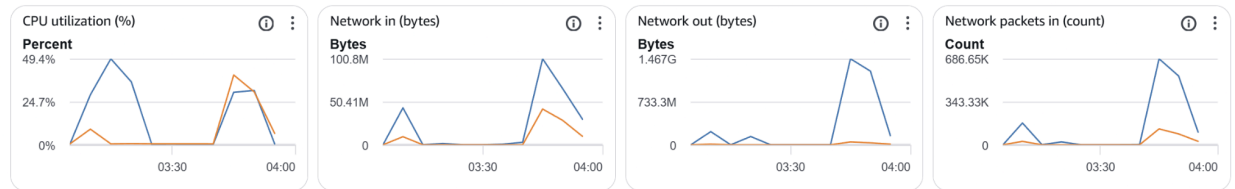
Room 15 | users: 50 | throughput: 5306.3 msg/s | mean latency: 118507ms | median latency: 120161ms
Message Types: JOIN: 1276 TEXT: 1204829 : 49 LEAVE: 1216

Room 8 | users: 50 | throughput: 5475.8 msg/s | mean latency: 118714ms | median latency: 112694ms
Message Types: JOIN: 1277 TEXT: 1224807 : 49 LEAVE: 1225

Room 6 | users: 50 | throughput: 5334.5 msg/s | mean latency: 119039ms | median latency: 121304ms
Message Types: JOIN: 1276 TEXT: 1230659 LEAVE: 1220

Room 10 | users: 50 | throughput: 5400.4 msg/s | mean latency: 115361ms | median latency: 114658ms
Message Types: JOIN: 1275 TEXT: 1243015 LEAVE: 1275

Room 9 | users: 50 | throughput: 5402.9 msg/s | mean latency: 120525ms | median latency: 120844ms
Message Types: LEAVE: 1270 JOIN: 1275 TEXT: 1236963
```



ALB + 2S | Pool=1000 | Workers=5/each | 500K msgs | Async Accept | InitialCredits=1000 (t3.small)



Sai Supriya Kotturu assignment-2

```
=== Performance Metrics ===
Total runtime (wall time): 328.3s
Successful messages sent: 582000
Failed messages: 0
Overall throughput: 1529.0 msg/s
Total connections: 1000
Reconnections: 0
Failed connections: 0
Mean Latency across all rooms: 152741ms
Median Latency across all rooms: 156417ms
95th Percentile Latency across all rooms: 163297ms
99th Percentile Latency across all rooms: 163836ms
Min Latency across all rooms: 49769ms
Max Latency across all rooms: 164465ms
Median Throughput across all rooms: 79.22 msg/s
Throughput across rooms:

Room 1 | users: 50 | throughput: 3999.7 msg/s | mean latency: 148161ms | median latency: 149414ms
Message Types: JOIN: 1875 TEXT: 1233104 LEAVE: 1275

Room 12 | users: 50 | throughput: 3954.5 msg/s | mean latency: 154589ms | median latency: 155265ms
Message Types: JOIN: 1748 TEXT: 1230610 LEAVE: 1275

Room 13 | users: 50 | throughput: 3933.7 msg/s | mean latency: 156468ms | median latency: 158258ms
Message Types: TEXT: 1229354 LEAVE: 1275 JOIN: 1648

Room 17 | users: 50 | throughput: 3948.2 msg/s | mean latency: 158057ms | median latency: 157001ms
Message Types: TEXT: 1228480 LEAVE: 1275 JOIN: 1727

Room 11 | users: 50 | throughput: 3979.4 msg/s | mean latency: 159472ms | median latency: 158046ms
Message Types: TEXT: 1224960 LEAVE: 1275 JOIN: 1652

Room 15 | users: 50 | throughput: 3966.1 msg/s | mean latency: 160493ms | median latency: 163235ms
Message Types: LEAVE: 1275 JOIN: 1660 TEXT: 1222333

Room 20 | users: 50 | throughput: 4257.3 msg/s | mean latency: 114253ms | median latency: 121274ms
Message Types: JOIN: 1768 TEXT: 1156151 LEAVE: 1273

Room 4 | users: 50 | throughput: 3974.9 msg/s | mean latency: 158858ms | median latency: 152531ms
Message Types: JOIN: 1847 TEXT: 1232088 LEAVE: 1275

Room 16 | users: 50 | throughput: 3932.9 msg/s | mean latency: 158377ms | median latency: 160589ms
Message Types: LEAVE: 1275 JOIN: 1798 TEXT: 1228933
```

```
Room 16 | users: 50 | throughput: 3932.9 msg/s | mean latency: 158377ms | median latency: 160589ms
Message Types: LEAVE: 1275 JOIN: 1798 TEXT: 1228933

Room 18 | users: 50 | throughput: 3963.6 msg/s | mean latency: 160376ms | median latency: 163143ms
Message Types: JOIN: 1677 TEXT: 1223100 LEAVE: 1275

Room 5 | users: 50 | throughput: 3937.8 msg/s | mean latency: 157436ms | median latency: 159127ms
Message Types: JOIN: 1769 TEXT: 1229386 LEAVE: 1275

Room 6 | users: 50 | throughput: 3947.6 msg/s | mean latency: 159878ms | median latency: 162067ms
Message Types: LEAVE: 1275 JOIN: 1695 TEXT: 1225695

Room 9 | users: 50 | throughput: 3952.6 msg/s | mean latency: 160058ms | median latency: 162458ms
Message Types: JOIN: 1753 TEXT: 1225080 LEAVE: 1275

Room 14 | users: 50 | throughput: 4122.2 msg/s | mean latency: 137976ms | median latency: 138615ms
Message Types: JOIN: 1901 TEXT: 1232143 LEAVE: 1275

Room 7 | users: 50 | throughput: 4089.9 msg/s | mean latency: 141357ms | median latency: 140482ms
Message Types: LEAVE: 1275 JOIN: 1891 TEXT: 1234042

Room 3 | users: 50 | throughput: 4030.3 msg/s | mean latency: 145357ms | median latency: 146325ms
Message Types: JOIN: 1892 TEXT: 1233511 LEAVE: 1275

Room 10 | users: 50 | throughput: 3973.2 msg/s | mean latency: 152722ms | median latency: 152374ms
Message Types: LEAVE: 1275 JOIN: 1756 TEXT: 1231900

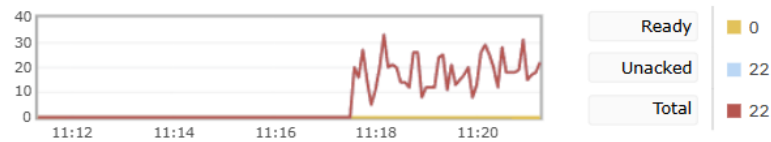
Room 19 | users: 50 | throughput: 3936.0 msg/s | mean latency: 159304ms | median latency: 161557ms
Message Types: TEXT: 1226896 LEAVE: 1275 JOIN: 1736

Room 8 | users: 50 | throughput: 3940.9 msg/s | mean latency: 159496ms | median latency: 161764ms
Message Types: LEAVE: 1275 JOIN: 1636 TEXT: 1225481

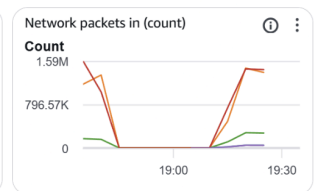
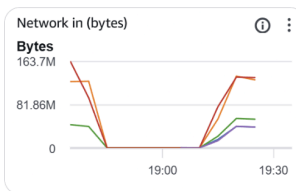
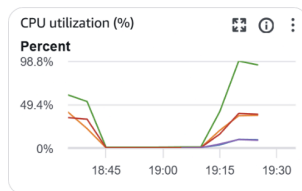
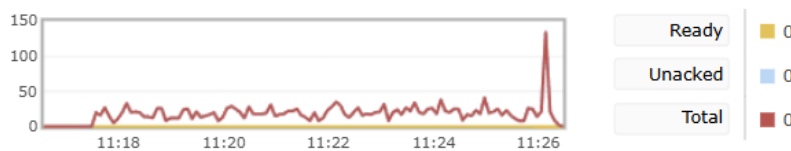
Room 2 | users: 50 | throughput: 3956.8 msg/s | mean latency: 160133ms | median latency: 162775ms
Message Types: JOIN: 1738 TEXT: 1224419 LEAVE: 1275
→ chat-flow git:(assignment-2) 📌 📌 📌
```

ALB + 4S | Pool=1000 | Workers=5/each | 500K msgs | Async Accept | InitialCredits=1000
(t3.small)

Queued messages last ten minutes ?



Queued messages last ten minutes ?



Sai Supriya Kotturu

assignment-2

```
=== Performance Metrics ===
Total runtime (wall time): 538.1s
Successful messages sent: 502000
Failed messages: 0
Overall throughput: 933.0 msg/s
Total connections: 1000
Reconnections: 0
Failed connections: 0
Mean Latency across all rooms: 254301ms
Median Latency across all rooms: 258805ms
95th Percentile Latency across all rooms: 267535ms
99th Percentile Latency across all rooms: 268033ms
Min Latency across all rooms: 78839ms
Max Latency across all rooms: 268416ms
Median Throughput across all rooms: 47.54 msg/s
Throughput across rooms:

Room 13 | users: 50 | throughput: 2367.3 msg/s | mean latency: 263432ms | median latency: 266626ms
Message Types: JOIN: 1872 TEXT: 1232649 LEAVE: 1275

Room 10 | users: 50 | throughput: 2375.1 msg/s | mean latency: 263618ms | median latency: 267189ms
Message Types: JOIN: 1879 TEXT: 1230911 LEAVE: 1275

Room 3 | users: 50 | throughput: 2430.2 msg/s | mean latency: 240853ms | median latency: 242488ms
Message Types: LEAVE: 1275 JOIN: 1850 TEXT: 1235185

Room 4 | users: 50 | throughput: 2377.2 msg/s | mean latency: 262395ms | median latency: 259911ms
Message Types: LEAVE: 1275 JOIN: 1875 TEXT: 1232330

Room 9 | users: 50 | throughput: 2382.0 msg/s | mean latency: 263013ms | median latency: 260316ms
Message Types: JOIN: 1868 TEXT: 1230745 LEAVE: 1275

Room 5 | users: 50 | throughput: 2456.5 msg/s | mean latency: 235063ms | median latency: 236160ms
Message Types: JOIN: 1884 TEXT: 1234638 LEAVE: 1275

Room 1 | users: 50 | throughput: 2367.6 msg/s | mean latency: 260058ms | median latency: 263395ms
Message Types: JOIN: 1865 TEXT: 1233786 LEAVE: 1275

Room 15 | users: 50 | throughput: 2369.2 msg/s | mean latency: 262491ms | median latency: 265757ms
Message Types: JOIN: 1878 TEXT: 1232685 LEAVE: 1275

Room 11 | users: 50 | throughput: 2374.6 msg/s | mean latency: 263066ms | median latency: 260096ms
Message Types: TEXT: 1231836 LEAVE: 1275 JOIN: 1888
```

```
Room 11 | users: 50 | throughput: 2374.6 msg/s | mean latency: 263066ms | median latency: 260096ms
Message Types: TEXT: 1231836 LEAVE: 1275 JOIN: 1888

Room 7 | users: 50 | throughput: 2370.8 msg/s | mean latency: 264059ms | median latency: 267526ms
Message Types: TEXT: 1231673 LEAVE: 1275 JOIN: 1861

Room 6 | users: 50 | throughput: 2381.6 msg/s | mean latency: 263289ms | median latency: 259970ms
Message Types: TEXT: 1229882 LEAVE: 1275 JOIN: 1879

254775ms
Message Types: LEAVE: 1275 JOIN: 1778 TEXT: 1234057

Room 16 | users: 50 | throughput: 2366.9 msg/s | mean latency: 259373ms | median latency: 261606ms
Message Types: JOIN: 1843 TEXT: 1235298 LEAVE: 1275

Room 18 | users: 50 | throughput: 2379.4 msg/s | mean latency: 258465ms | median latency: 257059ms
Message Types: JOIN: 1851 TEXT: 1234788 LEAVE: 1275

Room 20 | users: 50 | throughput: 2368.5 msg/s | mean latency: 262081ms | median latency: 265171ms
Message Types: JOIN: 1878 TEXT: 1232904 LEAVE: 1275

Room 19 | users: 50 | throughput: 2519.1 msg/s | mean latency: 198869ms | median latency: 208938ms
Message Types: JOIN: 1803 TEXT: 1165620 LEAVE: 1267

Room 12 | users: 50 | throughput: 2392.8 msg/s | mean latency: 249542ms | median latency: 251566ms
Message Types: LEAVE: 1275 JOIN: 1821 TEXT: 1234988

Room 8 | users: 50 | throughput: 2372.2 msg/s | mean latency: 260731ms | median latency: 258583ms
Message Types: JOIN: 1859 TEXT: 1232839 LEAVE: 1275
→ chat-flow git:(assignment-2) 🌟 🍷 📄
```